

Trabajo Práctico

TRABAJO INTEGRADOR

Alumnos - Grupo n° 280

Zaylene Gascon

Tecnicatura Universitaria en Programación
Universidad Tecnológica Nacional.

Programación 1

16 de [Junio] de 2026

Índice

Índice.....	2
1. Introducción y Objetivo.....	1
2. Marco Teórico	3
2.1. Listas	3
2.2. Diccionarios	3
2.3. Funciones	4
2.4. Estructuras condicionales	4
2.5. Estructuras repetitivas (bucles).....	4
2.6. Ordenamientos	5
2.7. Estadísticas básicas	5
2.8. Archivos CSV.....	5
3. Decisiones Técnicas y Arquitectura	5
3.1. Estructura de módulos.....	6
3.2. Diagrama de flujo de las operaciones principales.....	6
3.3. Validaciones y manejo de errores.....	7
3.4. Capturas de pantalla de la ejecución	7
4. Dificultades y Conclusiones	8
4.1. Dificultades encontradas	8
4.2. Conclusiones	9
5. Bibliografía y Webgrafía	9
6. Enlaces del Proyecto.....	9

1. Introducción y Objetivo

El presente Trabajo Práctico Integrador consiste en el desarrollo de una **aplicación de consola en Python** para la gestión de información de países. El sistema permite cargar un conjunto de datos desde un archivo CSV y operar sobre él mediante un menú interactivo: agregar y actualizar países, buscar, filtrar, ordenar y calcular estadísticas.

El **objetivo principal** es afianzar el uso de las estructuras de datos fundamentales (listas y diccionarios), la modularización del código mediante funciones (una función = una responsabilidad) y la aplicación de técnicas de filtrado y ordenamiento, integrando todos los conceptos vistos en Programación 1.

Cada país se modela como un **diccionario** con cuatro datos: nombre (texto), población (entero), superficie en km² (entero) y continente (texto). El conjunto de todos los países se almacena en una **lista**.

2. Marco Teórico

A continuación, se explican los conceptos teóricos aplicados en el proyecto, relacionándolos con su uso concreto en el sistema desarrollado.

2.1. Listas

Una **lista** es una estructura de datos que permite almacenar una colección **ordenada y mutable** de elementos. Cada elemento ocupa una posición (índice) que comienza en 0, y puede ser de cualquier tipo. Las listas admiten agregar, eliminar y modificar elementos durante la ejecución del programa (Python Software Foundation, 2024a).

En este proyecto, la lista es la estructura central: todos los países se guardan en una única lista sobre la cual se recorre, filtra y ordena. Por ejemplo, agregar un país equivale a hacer un `append()` sobre esa lista.

2.2. Diccionarios

Un **diccionario** es una estructura que almacena pares **clave-valor**. A diferencia de las listas, los elementos no se acceden por posición sino por su clave, lo que hace que la lectura del código sea más clara y semántica (Python Software Foundation, 2024a).

Cada país se representa con un diccionario, lo que permite acceder a sus datos por nombre de campo:

```
pais = {  
    "nombre": "Argentina",  
    "poblacion": 45376763,  
    "superficie": 2780400,  
    "continente": "América"  
}  
print(pais["poblacion"])    # 45376763
```

2.3. Funciones

Una **función** es un bloque de código reutilizable que realiza una tarea específica. Recibe datos a través de **parámetros** y puede devolver un resultado con `return`. Las funciones favorecen la **modularización**: dividir un problema grande en partes pequeñas y manejables, evitando repetir código (Python Software Foundation, 2024b).

Todo el sistema está construido sobre este principio: cada operación (buscar, filtrar, ordenar, calcular estadísticas) está aislada en su propia función dentro de un módulo, siguiendo la regla “una función = una responsabilidad”.

2.4. Estructuras condicionales

Las **estructuras condicionales** (`if`, `elif`, `else`) permiten que el programa tome decisiones y ejecute distintos bloques de código según se cumpla o no una condición lógica (Marzal, Gracia & García, 2014).

En el proyecto se utilizan, por ejemplo, para decidir qué opción del menú ejecutar, para validar que un dato no esté vacío, o para comparar si la población de un país entra dentro de un rango buscado.

2.5. Estructuras repetitivas (bucles)

Los **bucles** (`for` y `while`) permiten repetir un conjunto de instrucciones varias veces. El bucle `for` se usa para recorrer colecciones (como la lista de países), mientras que el `while` repite mientras se cumpla una condición (Marzal et al., 2014).

El menú principal funciona con un `while True` que se repite hasta que el usuario elige salir, y casi todas las operaciones recorren la lista de países con un `for`.

2.6. Ordenamientos

Ordenar consiste en reorganizar los elementos de una colección según un criterio (alfabético, numérico, ascendente o descendente). En el proyecto se implementó el algoritmo **burbuja (bubble sort)**, que compara pares de elementos contiguos y los intercambia cuando están en el orden incorrecto, repitiendo el recorrido hasta que la lista queda ordenada (Cormen, Leiserson, Rivest & Stein, 2009).

Se eligió implementarlo manualmente con fines didácticos, para evidenciar el funcionamiento interno de un ordenamiento. La alternativa nativa de Python sería la función `sorted()` con el parámetro `key`.

2.7. Estadísticas básicas

Las **estadísticas básicas** resumen un conjunto de datos en indicadores. En el sistema se calculan: máximo y mínimo (país con mayor y menor población), **promedio** (suma de los valores dividida por la cantidad de elementos) y **frecuencia** (cantidad de países por continente). Estos cálculos se realizan recorriendo la lista y acumulando resultados (Walpole, Myers, Myers & Ye, 2012).

2.8. Archivos CSV

Un archivo **CSV** (Comma-Separated Values) es un formato de texto plano donde cada línea representa un registro y los campos se separan por comas. Es ampliamente usado para intercambiar datos tabulares. Python incluye el módulo estándar `csv`, que facilita su lectura y escritura; en particular `DictReader` convierte cada fila en un diccionario usando los encabezados como claves (Python Software Foundation, 2024c).

El sistema lee el dataset inicial desde `países.csv` y guarda en él los cambios cada vez que se agrega o actualiza un país, logrando **persistencia** de los datos entre ejecuciones.

3. Decisiones Técnicas y Arquitectura

El proyecto se organizó en **módulos** (archivos `.py` independientes), aplicando el principio de separación de responsabilidades. Cada módulo agrupa funciones relacionadas, lo que facilita el mantenimiento, la lectura y el trabajo en equipo.

3.1. Estructura de módulos

Módulo	Responsabilidad
<code>main.py</code>	Bloque principal. Muestra el menú e integra los demás módulos coordinando el flujo del programa.
<code>datos.py</code>	Lectura y escritura del CSV, alta de países y actualización de población/superficie.
<code>validaciones.py</code>	Lectura segura de entradas: textos no vacíos, números válidos, rangos coherentes y control de duplicados.
<code>busquedas.py</code>	Búsqueda de países por nombre con coincidencia parcial o exacta.
<code>filtros.py</code>	Filtrado por continente y por rangos de población y superficie.
<code>ordenamiento.py</code>	Ordenamiento por nombre, población o superficie (ascendente/descendente) mediante bubble sort.
<code>estadisticas.py</code>	Cálculo de mayor/menor población, promedios y cantidad de países por continente.

3.2. Diagrama de flujo de las operaciones principales

El siguiente diagrama representa el flujo general del programa, desde la carga del dataset hasta la selección de operaciones en el menú y la salida.

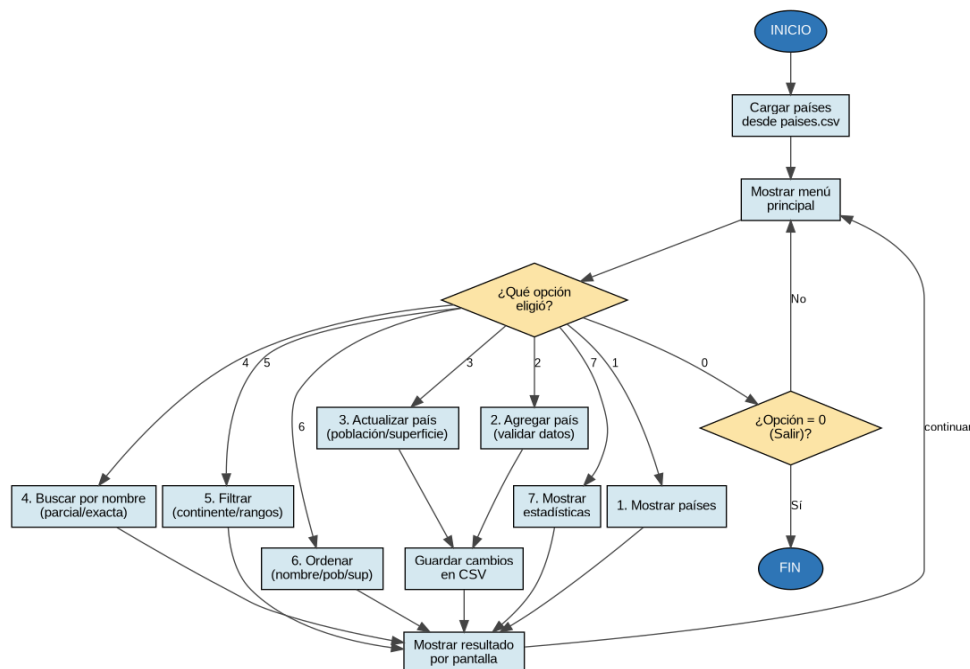


Figura 1. Diagrama de flujo del sistema.

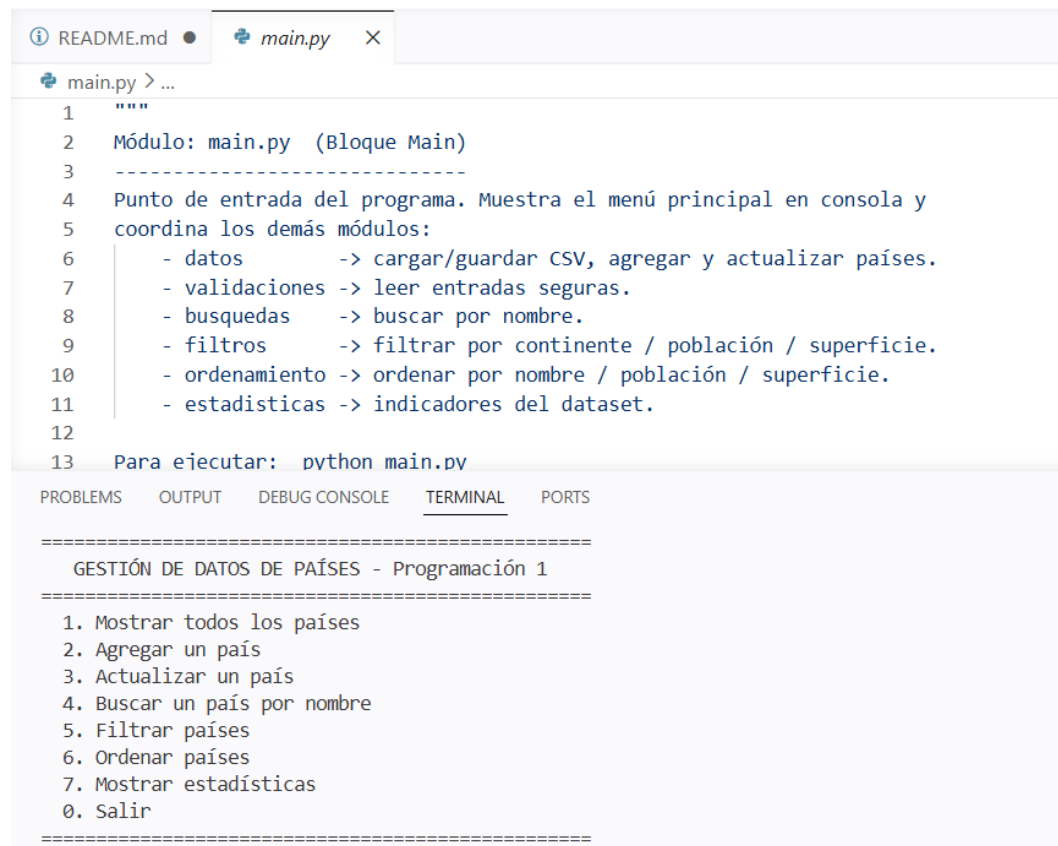
3.3. Validaciones y manejo de errores

- **Campos vacíos:** no se permite agregar un país con nombre o continente en blanco.
- **Tipos de datos:** población y superficie deben ser números enteros; si el usuario ingresa texto, se vuelve a pedir el dato.
- **CSV corrupto:** las filas con formato incorrecto o columnas faltantes se ignoran y se avisa por pantalla, sin frenar el programa (try/except).
- **Búsquedas/filtros sin resultados:** se muestra un mensaje claro en lugar de una tabla vacía o un error.
- **Duplicados:** no se permite cargar dos países con el mismo nombre.

3.4. Capturas de pantalla de la ejecución

A continuación se muestran ejemplos reales de la salida del programa.

Menú principal:



```

1  """
2  Módulo: main.py  (Bloque Main)
3  -----
4  Punto de entrada del programa. Muestra el menú principal en consola y
5  coordina los demás módulos:
6      - datos      -> cargar/guardar CSV, agregar y actualizar países.
7      - validaciones -> leer entradas seguras.
8      - busquedas  -> buscar por nombre.
9      - filtros    -> filtrar por continente / población / superficie.
10     - ordenamiento -> ordenar por nombre / población / superficie.
11     - estadísticas -> indicadores del dataset.
12
13  Para ejecutar:  python main.py

```

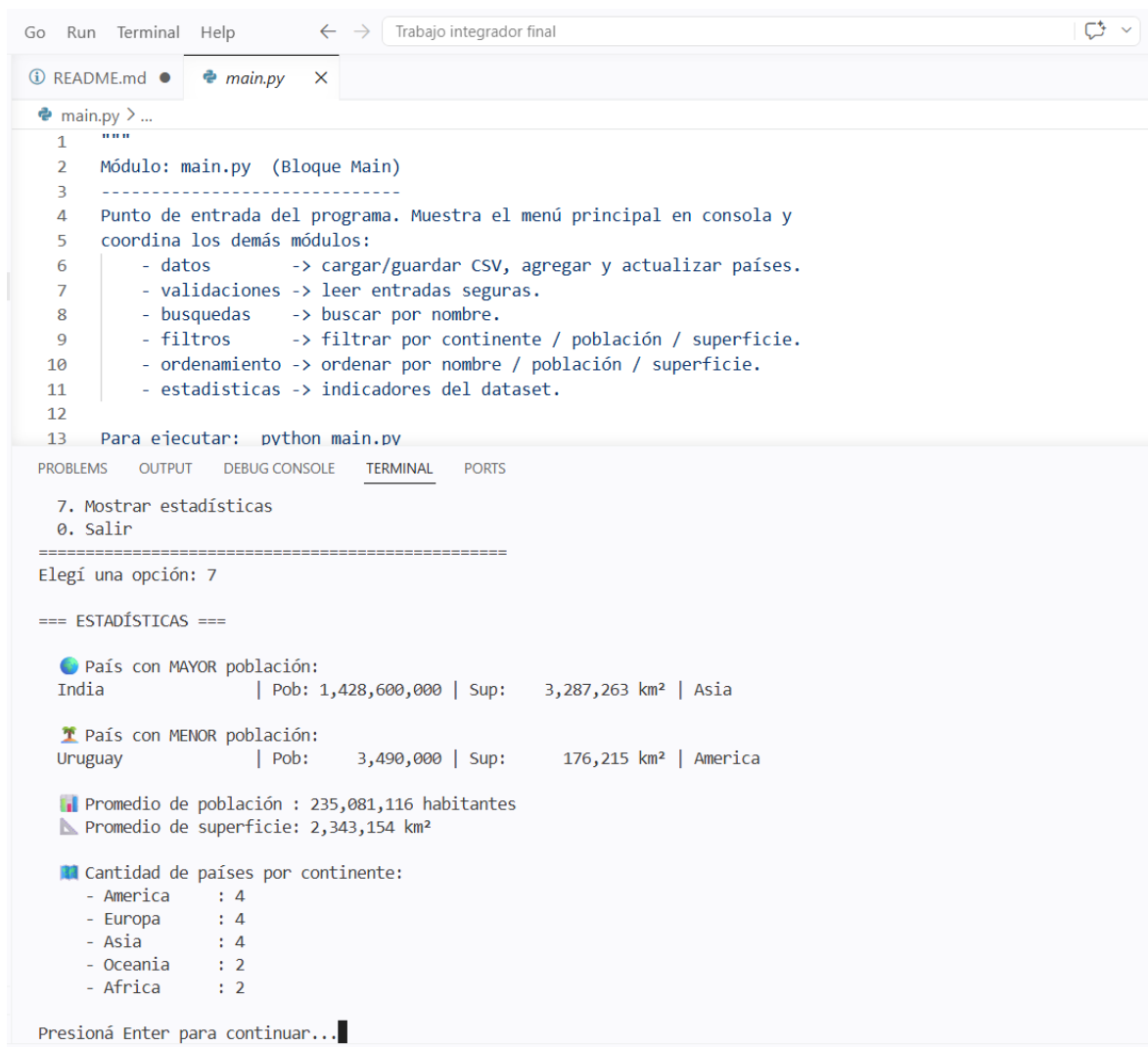
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

=====
GESTIÓN DE DATOS DE PAÍSES - Programación 1
=====
1. Mostrar todos los países
2. Agregar un país
3. Actualizar un país
4. Buscar un país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
0. Salir
=====

```

Estadísticas (opción 7):



```

1  """
2  Módulo: main.py (Bloque Main)
3  -----
4  Punto de entrada del programa. Muestra el menú principal en consola y
5  coordina los demás módulos:
6      - datos      -> cargar/guardar CSV, agregar y actualizar países.
7      - validaciones -> leer entradas seguras.
8      - busquedas  -> buscar por nombre.
9      - filtros    -> filtrar por continente / población / superficie.
10     - ordenamiento -> ordenar por nombre / población / superficie.
11     - estadísticas -> indicadores del dataset.
12
13  Para ejecutar: python main.py

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

7. Mostrar estadísticas
0. Salir
=====
Elegí una opción: 7

=== ESTADÍSTICAS ===

🌐 País con MAYOR población:
India | Pob: 1,428,600,000 | Sup: 3,287,263 km² | Asia

🌐 País con MENOR población:
Uruguay | Pob: 3,490,000 | Sup: 176,215 km² | America

🌐 Promedio de población : 235,081,116 habitantes
🌐 Promedio de superficie: 2,343,154 km²

🌐 Cantidad de países por continente:
- America : 4
- Europa : 4
- Asia : 4
- Oceania : 2
- Africa : 2

Presioná Enter para continuar...

```

4. Dificultades y Conclusiones

4.1. Dificultades encontradas

- **Lectura del CSV:** al principio el programa se rompía si una fila tenía un dato no numérico. Se resolvió encerrando la conversión de tipos en un bloque try/except e ignorando las filas inválidas.
- **Ordenamiento alfabético:** los nombres con mayúsculas se ordenaban antes que los de minúsculas. Se solucionó comparando los textos en minúsculas dentro del algoritmo.

- **Persistencia de los datos:** inicialmente los países agregados se perdían al cerrar el programa. Se incorporó el guardado automático en el CSV tras cada alta o actualización.
- **Organización en equipo:** coordinar quién trabajaba cada módulo. La modularización ayudó a dividir el trabajo sin pisarnos el código.

4.2. Conclusiones

El desarrollo del trabajo permitió integrar de forma práctica los conceptos centrales de Programación 1. Comprobamos que el uso de **diccionarios** para modelar entidades y de **listas** para colecciones hace el código más claro y cercano al problema real.

La **modularización** resultó clave: dividir el sistema en módulos con responsabilidades únicas facilitó probar cada parte por separado, encontrar errores y repartir el trabajo en equipo. Implementar el ordenamiento de forma manual nos ayudó a entender qué ocurre “por debajo” de una función como `sorted()`.

Finalmente, incorporar **validaciones y manejo de errores** nos mostró la diferencia entre un programa que funciona solo con datos perfectos y uno robusto, capaz de responder con mensajes claros ante entradas inesperadas.

5. Bibliografía y Webgrafía

Cormen, T. H., Leiserson, C. E., Rivest, R. L. & Stein, C. (2009). *Introduction to Algorithms* (3.^a ed.). MIT Press.

Marzal, A., Gracia, I. & García, P. (2014). *Introducción a la programación con Python 3*. Universitat Jaume I.

Python Software Foundation. (2024a). Data Structures — The Python Tutorial. <https://docs.python.org/3/tutorial/datastructures.html>

Python Software Foundation. (2024b). Defining Functions — The Python Tutorial. <https://docs.python.org/3/tutorial/controlflow.html>

Python Software Foundation. (2024c). csv — CSV File Reading and Writing. <https://docs.python.org/3/library/csv.html>

Walpole, R. E., Myers, R. H., Myers, S. L. & Ye, K. (2012). *Probabilidad y estadística para ingeniería y ciencias* (9.^a ed.). Pearson.

6. Enlaces del Proyecto

- **Repositorio GitHub:** <https://github.com/Zay016/TP-Integrador-Paises>
- **Video explicativo:** <https://youtu.be/CXHaFGJY8wE>